HAND GESTURE RECOGNITION

Using Convolutional Neural Networks (CNN)

PROJECT DOCUMENTATION

Project Type	Deep Learning / Image Classification
Dataset	LeapGestRecog
Development Platform	Google Colab
Author	Shafaat Ullah

Prepared as a technical overview of the dataset pipeline, CNN architecture, training, evaluation, and model outputs.

Table of Contents

• 1. Executive Summary

• 2. Project Objectives

• 3. Dataset Description

• 4. Tools and Technologies

• 5. System Workflow

• 6. Data Preparation

• 7. CNN Model Architecture

• 8. Training Strategy

• 9. Evaluation and Results

• 10. Prediction and Output Files

• 11. How to Run the Project

• 12. Limitations

• 13. Future Enhancements

• 14. Conclusion

Note: This document describes the project design and code workflow. Training metrics are intentionally left for the actual notebook results; no accuracy value is assumed or fabricated.

1. Executive Summary

Hand Gesture Recognition is a computer vision task that identifies a hand gesture from an image. This project uses a Convolutional Neural Network (CNN) implemented with TensorFlow and Keras to classify images from the LeapGestRecog dataset. The workflow includes downloading the dataset through Python, collecting image paths and labels, preprocessing images, splitting data into training, validation, and test sets, training the model, and evaluating its performance. The trained model can then be used to predict the class of a new hand image.

2. Project Objectives

• Download and inspect the LeapGestRecog dataset programmatically in Google Colab.

• Organize images and derive labels from their folder names.

• Resize and normalize images for consistent CNN input.

• Train a multi-class CNN classifier.

• Measure performance using test accuracy, loss, a classification report, and a confusion matrix.

• Visualize sample predictions and export results for further analysis.

• Save the trained model and class-name mapping for reuse.

3. Dataset Description

The project uses LeapGestRecog, a hand-gesture image dataset commonly used for gesture classification experiments. The dataset is downloaded using the KaggleHub Python package rather than requiring a manual ZIP upload. The notebook scans the downloaded directory recursively for PNG and JPG files and derives each image label from its parent folder. The exact number of images and classes should be confirmed from the output of the dataset-inspection cells.

Dataset source: https://www.kaggle.com/datasets/gti-upm/leapgestrecog

Data-splitting note: The notebook uses a stratified split: 70% training, 15% validation, and 15% testing. If multiple images of the same person or capture session appear across splits, the measured score may be optimistic. For a more rigorous experiment, split by subject or session where dataset metadata permits.

4. Tools and Technologies

Technology	Purpose
Python	Main programming language
Google Colab	Notebook environment and optional GPU runtime
KaggleHub	Programmatic dataset download
TensorFlow / Keras	CNN construction, training, and model saving
NumPy / Pandas	Numerical operations and image metadata tables
Matplotlib	Training plots and sample-image visualization
Scikit-learn	Stratified splitting and classification metrics
Pillow (PIL)	Optional image inspection and handling

5. System Workflow

Step	Stage	Description
1	Dataset download	Retrieve the dataset using KaggleHub.
2	Image discovery	Scan folders and create image-path / label records.
3	Data split	Create training, validation, and test subsets.
4	Preprocessing	Decode images, resize to 96 × 96, convert to grayscale, normalize pixels to 0–1.
5	Model training	Train the CNN using Adam and categorical class probabilities.
6	Evaluation	Review accuracy, loss, confusion matrix, and per-class metrics.
7	Prediction	Predict the gesture label and confidence for an image.
8	Export	Save the model, class labels, and CSV prediction results.

6. Data Preparation

Images are located by recursively searching the dataset directory for supported image extensions. Labels are derived from each image's immediate parent folder. The image pipeline then performs the following operations:

• Read image bytes from disk.

• Decode each image into a single grayscale channel.

• Resize the image to 96 × 96 pixels.

• Convert pixel values to floating-point numbers and scale them to the 0–1 range.

• Batch samples for efficient TensorFlow training and prefetch data where possible.

The class-name list is sorted and mapped to integer IDs. The model uses sparse categorical cross-entropy, which is appropriate when labels are integer class IDs.

7. CNN Model Architecture

The CNN extracts spatial patterns from hand images and maps those features to gesture classes. The implemented model uses the following high-level sequence:

Layer / Block	Role
Input: 96 × 96 × 1	Accepts grayscale image tensors
Conv2D: 32 filters + BatchNorm + MaxPooling	Learns low-level edges and shapes
Conv2D: 64 filters + BatchNorm + MaxPooling	Learns more complex local patterns
Conv2D: 128 filters + BatchNorm + MaxPooling	Builds higher-level visual features
Conv2D: 128 filters + MaxPooling	Further feature extraction
GlobalAveragePooling2D	Reduces spatial feature maps
Dense: 128 + ReLU + Dropout 0.4	Learns classification features and helps regularization
Dense: number of classes + Softmax	Outputs class probabilities

Compilation uses the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as a tracked metric.

8. Training Strategy

• Maximum epochs: 20.

• Batch size: 32.

• Optimizer: Adam with an initial learning rate of 0.001.

• Early stopping monitors validation loss and restores the best weights after the patience threshold.

• ReduceLROnPlateau lowers the learning rate when validation loss stops improving.

• ModelCheckpoint saves the model with the best validation accuracy.

Training may stop before the maximum epoch count if validation loss does not improve. A GPU runtime can reduce training time, but availability depends on the Colab account and current resource allocation.

9. Evaluation and Results

The notebook evaluates the final selected model on the held-out test subset. It reports test accuracy and test loss, plots training and validation accuracy/loss, and generates a classification report and confusion matrix. The confusion matrix helps identify which gesture categories are being confused with one another.

Metric	Value
Test accuracy	Fill in from Cell 13 output: _____ %
Test loss	Fill in from Cell 13 output: _____
Training images	Fill in from Cell 8 output: _____
Validation images	Fill in from Cell 8 output: _____
Testing images	Fill in from Cell 8 output: _____

Reporting guidance: Do not claim a specific accuracy until it has been measured from the actual run. Training accuracy alone is not a reliable estimate of performance on unseen data; use the held-out test metrics.

10. Prediction and Output Files

The prediction function applies the same grayscale conversion, resizing, and normalization used during training. The model returns class probabilities; the class with the highest probability is displayed as the predicted gesture, alongside a confidence score.

hand_gesture_recognition.keras — Saved TensorFlow/Keras model.

gesture_classes.json — Ordered list of class names matching the output indices.

hand_gesture_predictions.csv — Image path, actual label, predicted label, confidence, and correctness flag.

Accuracy and loss plots — Visual comparison of training and validation progress.

Confusion matrix and classification report — Evaluation of errors and per-class precision, recall, and F1-score.

11. How to Run the Project

1. Open Google Colab and create a new Python notebook.

2. Select a GPU runtime if one is available (Runtime → Change runtime type).

3. Run the KaggleHub installation and dataset-download cells.

4. Run the dataset inspection and image-collection cells; confirm that image paths and class labels are correct.

5. Run the train/validation/test split and TensorFlow input-pipeline cells.

6. Build the CNN and train it using the callbacks.

7. Run the graph, test evaluation, confusion matrix, prediction, and export cells.

8. Save the notebook using File → Download → Download .ipynb.

Dependencies

```
tensorflow, numpy, pandas, matplotlib, scikit-learn, Pillow, kagglehub
```

12. Limitations

• The described model predicts from a supplied image; real-time webcam recognition is not included in this workflow.

• Performance depends on image quality, class balance, preprocessing, and the training/test split.

• Folder-derived labels must be verified against the dataset's actual directory structure.

• A random image-level split can overestimate generalization when near-duplicate images or the same subjects occur in multiple subsets.

• Confidence values from softmax are not necessarily calibrated probabilities.

13. Future Enhancements

• Add live webcam recognition with OpenCV.

• Apply careful data augmentation to improve robustness.

• Use subject-wise or session-wise splits for a stronger evaluation.

• Compare the CNN with transfer-learning models such as MobileNetV2.

• Deploy the model in a simple web interface using Streamlit or Flask.

• Add additional gestures and test performance under different lighting and backgrounds.

14. Conclusion

This project provides an end-to-end foundation for hand gesture image classification using a CNN. It covers dataset acquisition, preprocessing, model training, quantitative evaluation, visualization, prediction, and saving reusable artifacts. The next step is to insert the actual test results from the completed Colab run and, if desired, extend the project to real-time webcam recognition.

Author: Shafaat Ullah

Project: Hand Gesture Recognition Using CNN | Dataset: LeapGestRecog | Platform: Google Colab